H2O provides both manual and automatic optimisation modes for faster and a more robust convergence of network parameters to data analysis and classification problems. To reduce oscillation during the convergence of network parameters, H2O uses the Learning Rate Annealing technique to reduce the learning rate as the network model approaches its goal.

H2O performs certain essential preprocessing of data. Other than categorical encoding, it also standardises data with respect to its activation functions. This is essential, as most of the activation functions generally do not map data into the full spectrum of the real numbers scale.

H2O and R

H2O supports standalone as well as R, Python and Web based interfaces. Here I shall discuss H2O in the R language platform. It is installed from the CRAN site with the *install.packages("h2o")* command from the command line. After successful installation, the package is loaded into the current workspace by the library (h2o) function call. Since H2O is a multi-core distributed system and can be loaded in a cluster of the system, it is invoked into the present computation environment by the *h2o.init()* command. In this case, to initialise the package into the local machine with all its available cores, *h2o.init(nthreads = -1)* is used. The '-1' indicates all the cores of the local host. By default, H2O uses two cores. In case H2O is installed in a server, the *h2o.init()* function can establish a connection between the local host and the remote server by specifying the server's IP address and port number as follows:

```
h2o.init(ip="172,16,8.90", port=5453)
```

Example: To demonstrate the strength and perfection of the deep learning approach here, I have taken up a problem related to optical character recognition (OCR). In general, OCR software first divides an alphabetic document into a grid containing a single character (glyph) in each cell. Then it compares each glyph with a set of all the characters to recognise the character of the glyph. The characters are then combined back into words and the system performs spelling and grammar checks as final processing.

The objective of this example is to identify each of the black-and-white rectangular pixels displayed as one of the 26 capital letters in the English alphabet. The glyph images are based on 20 different fonts and each letter within these 20 fonts has been randomly distorted to produce a file of 20,000 unique stimuli. Each stimulus has been converted into 16 primitive numerical attributes called statistical moments and edge counts, which have then been scaled to fit into a range of integer values from 0 through 15.

For this example, the first 16,000 items are taken for training of the neural model and then the trained model is used to predict the category for the remaining 4000 font-variations of the 26 letters of the English alphabet. The used data set is by W. Frey and D.J. Slat and is available from *http://archive.ics.uci.edu/ml/datasets/Letter+Recognition*. Each character is represented by a glyph and the task is to match each with one of the 26 English letters for their classification. There are 20,000 rows and 17 attributes of the character data set, as shown in Table 1.

Table 1: Attribute information

1.	letter	capital letter	(26 values from A to Z)
2.	x-box	horizontal position of box	(integer)
3.	y-box	vertical position of box	(integer)
4.	width	width of box	(integer)
5.	high	height of box	(integer)
6.	onpix	total # on pixels	(integer)
7.	x-bar	mean x of on pixels in box	(integer)
8.	y-bar	mean y of on pixels in box	(integer)
9.	x2bar	mean x variance	(integer)
10.	y2bar	mean y variance	(integer)
11.	xybar	mean x y correlation	(integer)
12.	x2ybr	mean of x * x * y	(integer)
13.	xy2br	mean of x * y * y	(integer)
14.	x-ege	mean edge count left to right	(integer)
15.	xegvy	correlation of x-ege with y	(integer)
16.	y-ege	mean edge count bottom to top (integer)	(integer)
17.	yegvx	correlation of y-ege with x	(integer)

Classification using H2O

The 16 attributes (2^{nd}-17^{th} rows) as stated in the above table measure different dimensional characteristics of the glyph (1st row)—the proportions of black versus white pixels, and the average horizontal and vertical position of the pixels, etc. We have to identify the glyph on the basis of these attributes, and then classify all the similar glyphs to one of the 26 characters.

To start with, first set the environment with the desired working directory and load the necessary libraries.

```
>path<- "I:\\DEEPNET"
>setwd(path)

>library(data.table)
>library(h2o)
>localH2o <- h2o.init(nthreads = -1)
```

Then download the *letters.csv* file from the above data archive. As per the requirements for H2O, the data set is then converted to the H2O data frame.

```
letterimage<- fread("letterdata.csv", stringsAsFactors = T)
```